ZKTrustLLM-Agents Level 4

Integrated Full Technical Documentation and Agentic AI Automation Roadmap

Generated: 2026-05-09 14:56

1. Executive Summary

This report integrates the earlier ZKTrustLLM-Agents L4 foundation with the later MCP/A2A, reference-bound proof, media-plane, DTLS-RTP, impairment, and Linux network namespace validation steps. It is intended as a supervisor-ready full-picture document and as a planning base for the next stage of agentic AI automation.

2. Full Project Picture

Layer	Implemented Work	Research Meaning
Control-plane semantics	Roles, policies, actions, trust states, gateway schema	Defines the bounded meaning of L4 decisions
Contract state	AgentRegistry, CapabilityManager, PolicyRegistry	Creates authenticated control-plane state
Proof pipeline	AUTH_V2, AUTH_V2.1, AUTH_V2.2	Enables Groth16 proof-governed admissible decisions
MCP/A2A coordination	MCP server, A2A references, reference bundle verification	Allows agents to coordinate through compact authenticated references
Control telemetry	Semi-live MCP/A2A latency and multi-agent scaling	Measures coordination cost and scalability
Media-plane baseline	Live RTP telemetry	Connects control-plane research to packet-level media delivery
Secure media-plane	DTLS-wrapped RTP validation	Adds protected media delivery baseline
Network stress	Loopback tc netem impairment	Measures behaviour under delay, jitter, and loss
Namespace realism	Two Linux network namespaces over veth	Moves closer to two-node deployment while staying reproducible
Automation roadmap	Validator, orchestrator, report compiler, feedback tracker	Prepares full agentic AI-assisted research workflow

3. Key Implementation Timeline

Phase	Main Work
Foundation	L4 control-plane architecture, contracts, gateway validation
AUTH_V2	Request-binding Groth16 proof
AUTH_V2.1	Policy-admissibility-aware proof
AUTH_V2.2	Trust-state/action admissible pair proof for Restricted -> Isolate
Steps 77-83	MCP/A2A reference coordination, negative tests, deterministic KPI evaluation
Steps 84-88	Journal write-up, figures, semi-live control telemetry, multi-agent scaling, report PDF
Step 89	Live RTP media-plane telemetry
Step 90	DTLS-wrapped RTP validation
Step 91	Plain RTP vs DTLS-RTP impairment matrix
Step 92	Updated technical report through Step 91
Step 93	Linux network namespace sender-receiver validation
Step 94	Full technical documentation through Step 93
Step 95	Integrated history and automation roadmap

4. Current Research Claim

The project now supports a coherent full-stack Level 4 prototype in which proof-governed agent decisions can be represented through contract-backed control-plane state, exchanged between agents through MCP/A2A reference coordination, measured through semi-live control-plane KPIs, and connected to RTP/DTLS media-plane behaviour under controlled network impairment.

5. Recommended Next Step

The next engineering step should be Step 96: a full result validator. This validator should check JSON validity, CSV existence, packet counts, bitrate, jitter, packet loss, Git cleanliness, and the presence of current Markdown/PDF reports. This will make future agentic automation safer.

Appendix A: Earlier Foundation AUTH_V2 to AUTH_V2.2

ZKTrustLLM-Agents L4 Earlier Foundation: AUTH_V2 to AUTH_V2.2

Purpose

This document incorporates the earlier implementation history into the current Level 4 project documentation.

The earlier foundation established the control-plane, gateway, contract, and proof pipeline that later Steps 77-93 build upon.

Earlier Project Foundation

The earlier L4 work implemented:

• documented Level 4 control-plane architecture,

• Solidity control-plane contracts,

• gateway-assisted validation,

• AUTH_V2 Groth16 proof path,

• AUTH_V2.1 policy-admissibility proof path,

• AUTH_V2.2 trust-state-aware admissible-action proof path,

• positive proof verification and on-chain decision submission,

• negative rejection flows for tampered or invalid proofs,

• stable Git checkpointing for safe continuation.

Core Contracts

The earlier contract layer included:

```
| Contract | Role |
|---|---|
| AgentRegistry.sol | Registers and manages active/suspended/revoked agents |
| CapabilityManager.sol | Issues and validates short-lived bounded capabilities |
| PolicyRegistry.sol | Stores role-policy and trust-state/action admissibility rules |
| DecisionAttestorAuthV2.sol | Stores AUTH_V2 proof-backed decisions |
| DecisionAttestorAuthV2_1.sol | Stores AUTH_V2.1 policy-admissible decisions |
| DecisionAttestorAuthV2_2.sol | Stores AUTH_V2.2 trust-state-aware decisions |
```

Gateway Validation

The gateway validation stage checked:

• whether the agent is registered,

• whether the capability is valid,

• whether the role-policy pairing is allowed,

• whether the trust-state/action pairing is admissible.

The positive validation path accepted:

• agent: `policy-lkh-01`

• policy class: `policy-enforce`

• action class: `isolate`

• trust state: `Restricted`

The negative validation path rejected role-policy mismatch using:

• `REJECT_POLICY_ROLE`

AUTH_V2

AUTH_V2 introduced the first control-plane-aware proof binding.

It bound:

• `agentKey`

• `capabilityId`

• `policyClassHash`

• `actionClass`

• `contextHash`

• `traceCommitment`

• `expiryBucket`

with private witness:

• `bindingNonce`

AUTH_V2.1

AUTH_V2.1 added policy admissibility.

New public field:

• `policyAdmissibilityFlag`

Core condition:

• `policyAdmissibilityFlag == 1`

This made the proof explicitly policy-aware.

AUTH_V2.2

AUTH_V2.2 added trust-state awareness.

New public field:

• `trustState`

Implemented admissible pair:

• `Restricted -> Isolate`

Core constraints:

• `policyAdmissibilityFlag == 1`

• `trustState == 3`

• `actionClass == 3`

Strongest Earlier Claim

The strongest earlier implementation claim was:

The repository supports trust-state-aware, policy-admissibility-aware, bounded Groth16 decision submission for the concrete admissible path `Restricted -> Isolate`.

Why This Matters for the Current Project

The later Steps 77-93 build on this foundation.

The earlier work provides:

• proof-governed decision semantics,

• on-chain attestation,

• bounded trust-state/action logic,

• contract-backed authorization context,

• reproducible checkpointing.

The later work extends this into:

- MCP context retrieval,
- A2A reference-based coordination,
- reference-bound proof validation,
- semi-live control-plane telemetry,
- multi-agent scaling,
- live RTP media-plane telemetry,
- DTLS-wrapped RTP validation,
- network impairment testing,
- Linux network namespace validation.

Appendix B: Full Project Timeline

ZKTrustLLM-Agents L4 Full Project Timeline

Phase 1: Architecture and Control-Plane Semantics

The project began by defining a proof-governed Level 4 agentic control plane.

Core concepts:

• agent identity,

• capability-bounded authorization,

• policy classes,

• action classes,

• trust states,

• gateway validation,

• proof-backed decision attestation.

Phase 2: Solidity Control-Plane Foundation

Implemented contracts:

• AgentRegistry.sol

• CapabilityManager.sol

• PolicyRegistry.sol

This created the contract-backed control-plane state.

Phase 3: Gateway Validation

The gateway demonstrated off-chain request validation against on-chain control-plane state.

This connected:

• registered agents,

• valid capabilities,

• admissible policies,

• trust-state/action rules.

Phase 4: AUTH_V2 to AUTH_V2.2 Proof Pipeline

AUTH_V2 introduced request binding.

AUTH_V2.1 added policy admissibility.

AUTH_V2.2 added trust-state/action admissibility for:

• Restricted -> Isolate

This established proof-governed on-chain decision submission.

Phase 5: MCP/A2A and Reference-Based Coordination

Later work extended the system from proof-backed single decisions to structured multi-agent coordination.

Key additions:

• A2A reference-aware coordination,

• MCP context server,

• reference bundle verification,

• Level 4 KPI summary,

• reference-bound proof direction.

Phase 6: Security and Negative Tests

Negative-security testing validated that invalid or tampered proof/control paths are rejected.

This strengthened the trustworthiness of the implementation.

Phase 7: Deterministic and Semi-Live Control-Plane Evaluation

The project then moved from purely deterministic evaluation toward measured control-plane telemetry.

Key outputs:

• deterministic network KPI analysis,

• benchmark figures,

• semi-live MCP/A2A control telemetry,

• multi-agent scaling telemetry.

Phase 8: Media-Plane Validation

The project then connected the control-plane research to multimedia delivery.

Implemented:

• live RTP media-plane telemetry,

• DTLS-wrapped RTP tunnel/proxy validation,

• plain RTP vs DTLS-RTP comparison.

Phase 9: Network Impairment Evaluation

The project introduced controlled impairment using Linux `tc netem`.

Measured:

• packet loss,

• bitrate,

• arrival-gap jitter,

• p50 jitter,

• p95 jitter.

Phase 10: Linux Network Namespace Validation

Step 93 reproduced impairment testing using two Linux network namespaces connected by a veth pair.

This improved realism compared with localhost-only loopback.

Current Full-Picture Claim

The project now demonstrates a staged Level 4 research prototype where proof-governed MCP/A2A control-plane coordination can be linked to secure RTP/DTLS media-plane behaviour under controlled network stress.

Appendix C: Agentic AI Automation Roadmap

ZKTrustLLM-Agents L4 Agentic AI Automation Roadmap

Purpose

This roadmap defines how the project can move from manual experiment execution to a structured agentic AI automation workflow.

The goal is not to remove human supervision, but to make experiments repeatable, measurable, auditable, and easier to report to supervisors.

Automation Layer 1: Experiment Orchestrator

Create one master runner that can execute:

• MCP/A2A control-plane tests

• reference bundle verification

• semi-live control telemetry

• multi-agent scaling telemetry

• live RTP media telemetry

• DTLS-RTP media telemetry

• loopback impairment matrix

• Linux namespace impairment matrix

Target script:

```
scripts/l4/run_l4_full_experiment_suite.py ■[200~Automation Layer 2: Result Validator Create a validator that checks every output file. Validation checks: JSON files are valid required CSV files exist packet counts are greater than zero bitrate is positive packet loss is within expected range jitter is not unrealistic Git working tree is clean before commit report PDF exists~Automation Layer 3: Report Compiler Create a single report compiler that automatically reads latest result JSON/CSV files and updates: Markdown technical report PDF technical report paper results tables artifact index Automation Layer 4: Supervisor Feedback Tracker Maintain a structured tracker for supervisor comments. Fields: feedback item source meeting/date affected section implementation step status evidence artifact next action ■[200~Automation Layer 5: Safe Agentic Planner Introduce an agentic planning layer that proposes next experiments but does not run privileged commands automatically. The planner can recommend: next experiment expected output files validation rules paper section to update likely supervisor-facing contribution Human approval should still be required for: sudo commands tc netem impairment namespace creation/deletion Git push deleting generated results Recommended Next Automation Steps StepGoal Step 96Build full result validator Step 97Build master experiment suite runner Step 98Build supervisor feedback tracker automation Step 99Build auto report compiler Step 100Run two-machine, Mininet, or ns-3 validation~ --- ## Step 95 Fix C: create the missing supervisor feedback tracker
```

mkdir -p docs/progress

cat > docs/progress/SUPERVISOR_FEEDBACK_TRACKER.md <<'EOF'

Supervisor Feedback Tracker

Purpose

This tracker maps supervisor feedback to implemented technical steps and remaining work.

```
| Feedback / Requirement | Implemented Evidence | Status | Next Action |

|---|---|---|---|

| Explain MCP/A2A over Level 4 clearly | Steps 79-87 documentation and telemetry | Done | Polish journal wording |

| Provide agent KPIs | Step 86 semi-live control telemetry and Step 87 multi-agent scaling | Done | Add repeated runs and confidence intervals later |

| Provide network KPIs | Steps 89-93 RTP, DTLS-RTP, impairment, and namespace results | Done | Repeat on two-machine or testbed setup |

| Move beyond deterministic emulation | Step 86 semi-live control-plane and Step 89 live RTP | Done | Extend to distributed topology |

| Add secured media validation | Step 90 DTLS-wrapped RTP validation | Done | Later compare against DTLS-SRTP/WebRTC-style design |
```

| Test under impairment | Step 91 loopback netem and Step 93 namespace netem | Done | Reproduce with Mininet, ns-3, or physical testbed |

| Provide full documentation | Step 94 and Step 95 reports | In progress | Generate expanded integrated PDF |

| Move toward full automation | Step 95 automation roadmap | In progress | Implement Step 96 result validator |

Current Supervisor-Facing Summary

The project now has a coherent full-stack path:

1. L4 control-plane semantics. 2. Contract-backed agent/capability/policy state. 3. AUTH_V2 to AUTH_V2.2 proof-governed admissibility. 4. MCP/A2A reference-based coordination. 5. Semi-live control-plane telemetry. 6. Multi-agent scaling telemetry. 7. Live RTP media-plane validation. 8. DTLS-wrapped RTP security layer. 9. Network impairment evaluation. 10. Namespace-based sender-receiver validation. 11. Full documentation and automation roadmap.

Current Research Direction

The next stage should move from manual experiments toward controlled automation.

Recommended next step:

• Step 96: implement a full result validator.

• Step 97: implement a full experiment-suite runner.

• Step 98: automate supervisor feedback tracking.

• Step 99: automate report generation.

• Step 100: repeat media-plane validation in a two-machine, Mininet, ns-3, or university testbed environment.

Appendix D: Supervisor Feedback Tracker

Missing file: /home/sk02352/zktrustllm-agents-l4/docs/progress/SUPERVISOR_FEEDBACK_TRACKER.md